

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

print("Everything is working!")
```

Everything is working!

```
import pandas as pd

data = pd.read_csv("student-mat.csv", sep=";")
```

```
data.head()
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	fr
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	4	
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	5	
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	4	
3	GP	F	15	U	GT3	T	4	2	health	services	...	3	
4	GP	F	16	U	GT3	T	3	3	other	other	...	4	

5 rows × 33 columns

```
data.shape
```

(395, 33)

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   school          395 non-null    object
1   sex             395 non-null    object
2   age             395 non-null    int64
3   address         395 non-null    object
4   famsize         395 non-null    object
5   Pstatus         395 non-null    object
6   Medu            395 non-null    int64
7   Fedu            395 non-null    int64
8   Mjob            395 non-null    object
9   Fjob            395 non-null    object
10  reason          395 non-null    object
11  guardian        395 non-null    object
12  traveltime      395 non-null    int64
13  studytime       395 non-null    int64
14  failures        395 non-null    int64
15  schoolsup       395 non-null    object
16  famsup          395 non-null    object
17  paid            395 non-null    object
18  activities      395 non-null    object
```

```

19  nursery      395 non-null    object
20  higher       395 non-null    object
21  internet     395 non-null    object
22  romantic     395 non-null    object
23  famrel       395 non-null    int64
24  freetime     395 non-null    int64
25  goout        395 non-null    int64
26  Dalc         395 non-null    int64
27  Walc         395 non-null    int64
28  health       395 non-null    int64
29  absences     395 non-null    int64
30  G1           395 non-null    int64
31  G2           395 non-null    int64
32  G3           395 non-null    int64
dtypes: int64(16), object(17)
memory usage: 102.0+ KB

```

```
data.describe()
```

	age	Medu	Fedu	traveltime	studytime	failures	famrel	freet
count	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000
mean	16.696203	2.749367	2.521519	1.448101	2.035443	0.334177	3.944304	3.235
std	1.276043	1.094735	1.088201	0.697505	0.839240	0.743651	0.896659	0.998
min	15.000000	0.000000	0.000000	1.000000	1.000000	0.000000	1.000000	1.000
25%	16.000000	2.000000	2.000000	1.000000	1.000000	0.000000	4.000000	3.000
50%	17.000000	3.000000	2.000000	1.000000	2.000000	0.000000	4.000000	3.000
75%	18.000000	4.000000	3.000000	2.000000	2.000000	0.000000	5.000000	4.000
max	22.000000	4.000000	4.000000	4.000000	4.000000	3.000000	5.000000	5.000

```
data["Pass"] = (data["G3"] >= 10).astype(int)
```

```
data[["G3", "Pass"]].head(10)
```

	G3	Pass
0	6	0
1	6	0
2	10	1
3	15	1
4	10	1
5	15	1
6	11	1
7	6	0
8	19	1
9	15	1

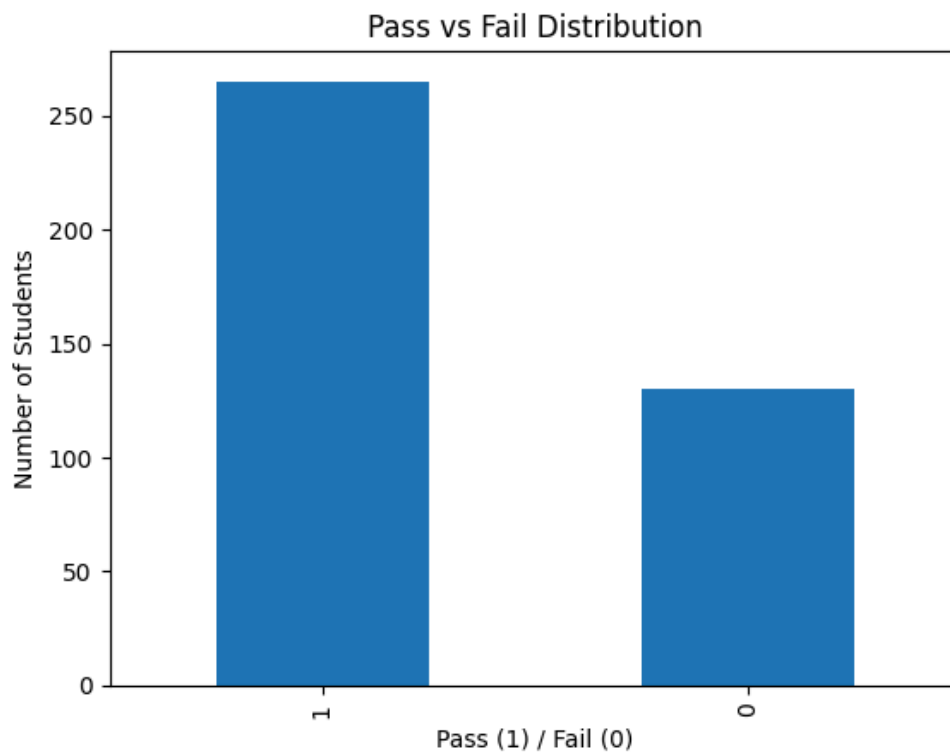
```
data["Pass"].value_counts()
```

	count
Pass	
1	265
0	130

dtype: int64

```
import matplotlib.pyplot as plt
```

```
data["Pass"].value_counts().plot(kind="bar")  
plt.title("Pass vs Fail Distribution")  
plt.xlabel("Pass (1) / Fail (0)")  
plt.ylabel("Number of Students")  
plt.show()
```



```
study_pass = data.groupby("studytime")["Pass"].mean()  
study_pass
```

Pass

	studytime
--	-----------

1	0.647619
---	----------

2	0.646465
---	----------

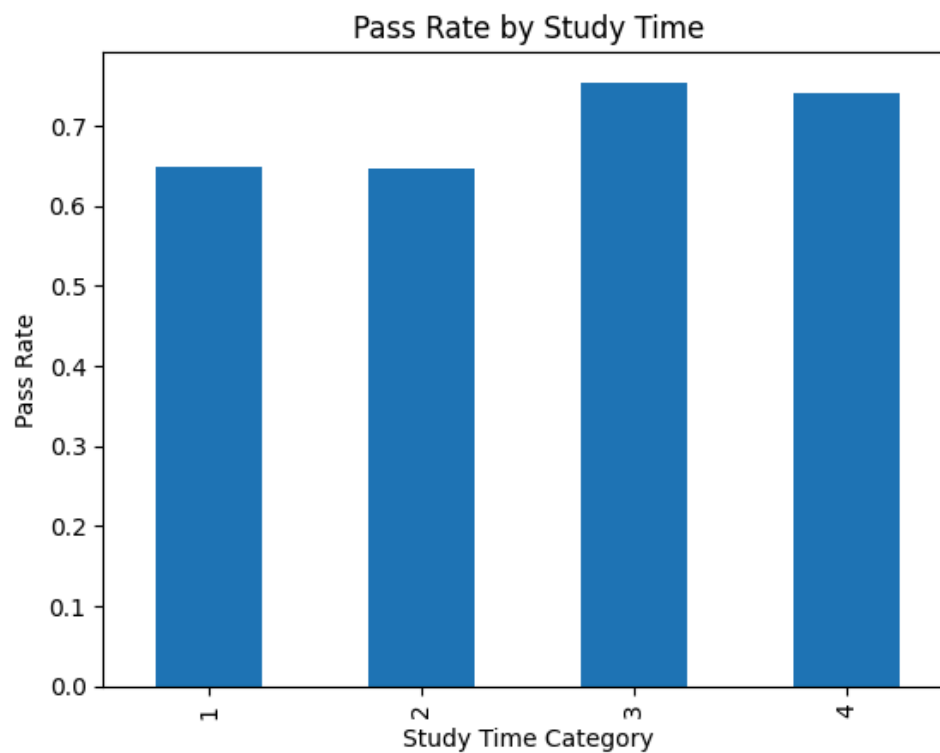
3	0.753846
---	----------

4	0.740741
---	----------

dtype: float64

```
study_pass.plot(kind="bar")
```

```
plt.title("Pass Rate by Study Time")  
plt.xlabel("Study Time Category")  
plt.ylabel("Pass Rate")  
plt.show()
```



```
failure_pass = data.groupby("failures")["Pass"].mean()  
failure_pass
```

Pass

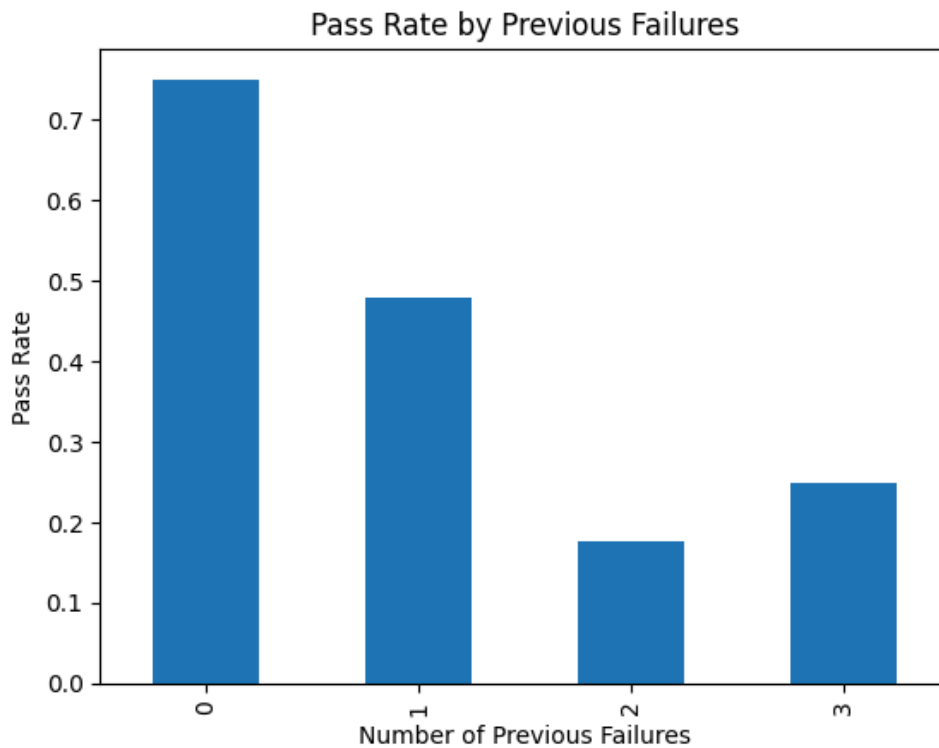
failures

0	0.750000
1	0.480000
2	0.176471
3	0.250000

dtype: float64

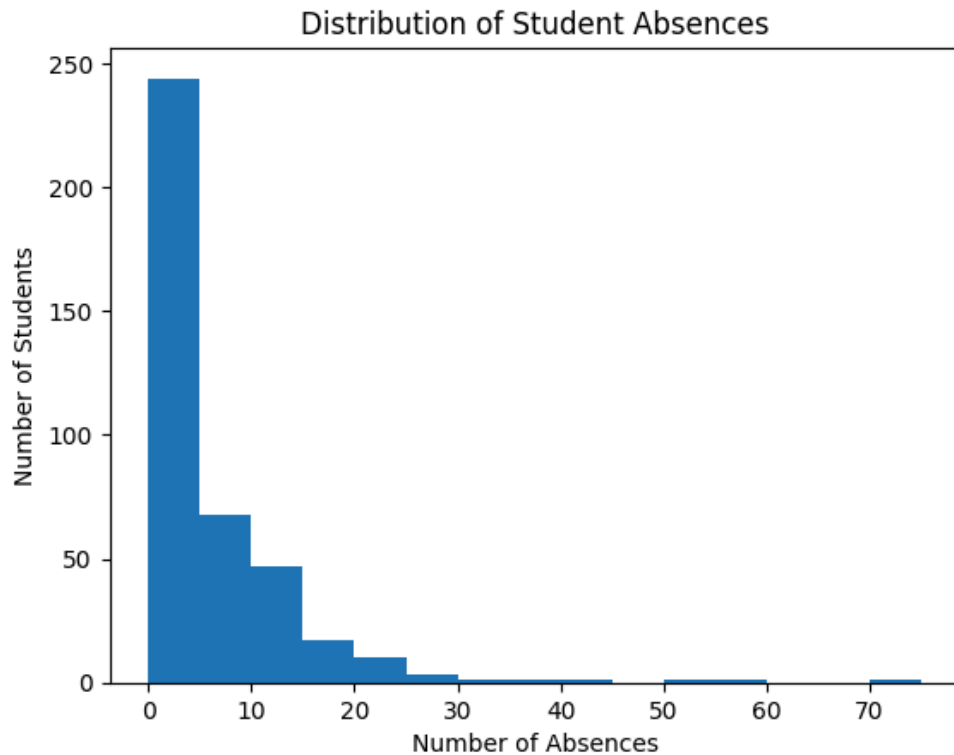
```
failure_pass.plot(kind="bar")

plt.title("Pass Rate by Previous Failures")
plt.xlabel("Number of Previous Failures")
plt.ylabel("Pass Rate")
plt.show()
```



```
data["absences"].plot(kind="hist", bins=15)

plt.title("Distribution of Student Absences")
plt.xlabel("Number of Absences")
plt.ylabel("Number of Students")
plt.show()
```



```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
data_ml = data.copy()

label_encoder = LabelEncoder()

for column in data_ml.columns:
    if data_ml[column].dtype == "object":
        data_ml[column] = label_encoder.fit_transform(data_ml[column])
```

```
X = data_ml.drop(["G3", "Pass"], axis=1)
y = data_ml["Pass"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

```
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)
```

```
▼ LogisticRegression ⓘ ?
LogisticRegression(max_iter=1000)
```

```
predictions = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, predictions)
```

```
print("Model Accuracy:", accuracy)
```

```
Model Accuracy: 0.9493670886075949
```

```
print(confusion_matrix(y_test, predictions))
```

```
[[26  1]
 [ 3 49]]
```

```
print(classification_report(y_test, predictions))
```

	precision	recall	f1-score	support
0	0.90	0.96	0.93	27
1	0.98	0.94	0.96	52
accuracy			0.95	79
macro avg	0.94	0.95	0.94	79
weighted avg	0.95	0.95	0.95	79

```
predictions = model.predict(X_test)
```

```
from sklearn.metrics import accuracy_score
```

```
accuracy = accuracy_score(y_test, predictions)
```

```
print("Model Accuracy:", accuracy)
```

```
Model Accuracy: 0.9493670886075949
```

```
from sklearn.metrics import confusion_matrix
```

```
cm = confusion_matrix(y_test, predictions)
```

```
print(cm)
```

```
[[26  1]
 [ 3 49]]
```

```
from sklearn.metrics import classification_report
```

```
print(classification_report(y_test, predictions))
```

	precision	recall	f1-score	support
0	0.90	0.96	0.93	27
1	0.98	0.94	0.96	52
accuracy			0.95	79
macro avg	0.94	0.95	0.94	79
weighted avg	0.95	0.95	0.95	79

```
import matplotlib.pyplot as plt

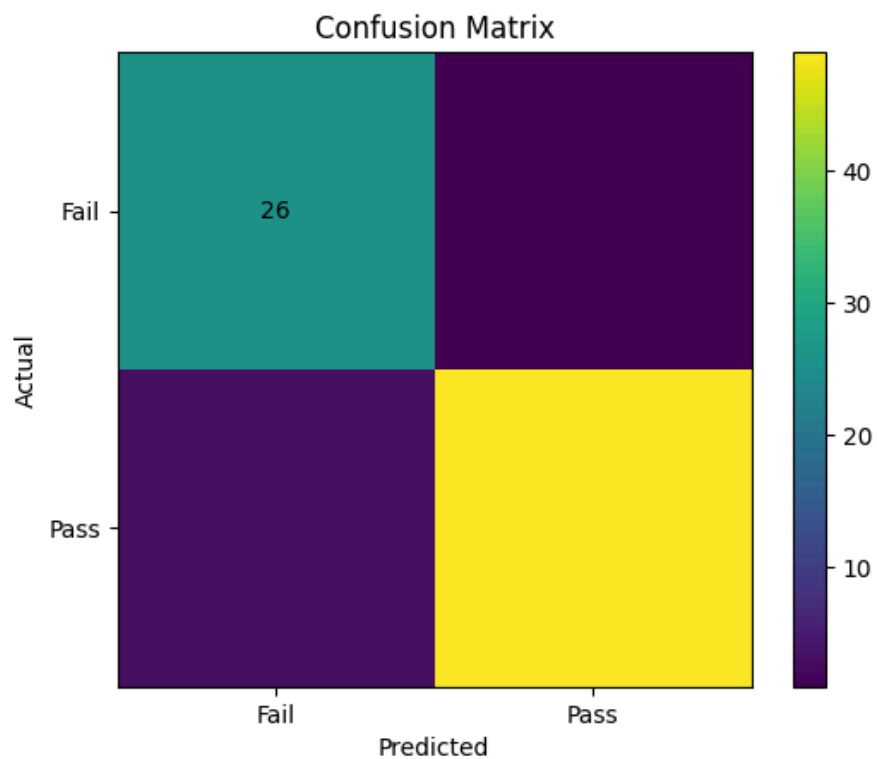
plt.imshow(cm, interpolation='nearest')
plt.title("Confusion Matrix")
plt.colorbar()

plt.xticks([0, 1], ["Fail", "Pass"])
plt.yticks([0, 1], ["Fail", "Pass"])

plt.xlabel("Predicted")
plt.ylabel("Actual")

for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, str(cm[i, j]), ha="center", va="center")

plt.show()
```

```
from sklearn.tree import DecisionTreeClassifier
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
dt_predictions = dt_model.predict(X_test)
from sklearn.metrics import accuracy_score
dt_accuracy = accuracy_score(y_test, dt_predictions)
print("Decision Tree Accuracy:", dt_accuracy)
```

1.03
Decision Tree Accuracy: 0.8860759493670886

```
from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
rf_predictions = rf_model.predict(X_test)
rf_accuracy = accuracy_score(y_test, rf_predictions)
```